

Homework 1

DEEP LEARNING: CS/DS 541

CHAD HUCEY | PEGAH EMDAD

Problem 3 - Linear Regression via Gradient Descent

$$a) f_{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$\text{where } \hat{y} = b + w_1 x_1 + \dots + w_m x_m = b + \sum_{i=1}^m w_i x_i$$

$$\Rightarrow \frac{\delta f_{MSE}}{\delta w_j} = \frac{2x_j}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

$$b) \frac{\delta f_{MSE}}{\delta w} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

$$= \frac{2}{n} x^T (\hat{y} - y)$$
$$= \frac{2}{n} (\hat{y} - y)^T x$$

$$c) \text{ Learning Rate} = 2 \times 10^{-4}$$

$$\text{Training Loss } D_{tr} = 297.1$$

$$\text{Test Loss } D_{te} = 312.3$$

d) Change the gradient descent function to also track loss on the test set. Stop gradient descent if the test loss stops decreasing.

Problem 4: The necessity of Non-linear activations

- 3 layer neural network with out activation function

input $\rightarrow$ vector x
 first hidden layer $\rightarrow W^{[1]} b^{[1]}$
 output $\rightarrow W^{[2]} b^{[2]}$

x shape $(1, m)$
 W shape (m, k)
 b shape $(1, k)$

$$a) \quad z^{[1]}_{(1, k)} = x_{(1, m)} W^{[1]}_{(m, k)} + b^{[1]}_{(1, k)}$$

$$b) \quad \hat{y}_{(1, q)} = z^{[1]}_{(1, k)} W^{[2]}_{(k, q)} + b^{[2]}_{(1, q)}$$

$$c) \quad \text{from part a)} \quad \hat{y} = z^{[1]} W^{[2]} + b^{[2]} \quad \xrightarrow{z^{[1]} = x W^{[1]} + b^{[1]}}$$

$$\begin{aligned} \hat{y} &= (x W^{[1]} + b^{[1]}) W^{[2]} + b^{[2]} \\ &= x W^{[1]} W^{[2]} + b^{[1]} W^{[2]} + b^{[2]} \\ &= x_{(1, m)} W^{[1]}_{(m, k)} W^{[2]}_{(k, q)} + b^{[1]}_{(1, k)} W^{[2]}_{(k, q)} + b^{[2]}_{(1, q)} \end{aligned}$$

can be written as $\hat{y} = W' x + b'$

d) Show above expression

$$\begin{aligned} \rightarrow \hat{y} &= (x W^{[1]} + b^{[1]}) W^{[2]} + b^{[2]} \\ \hat{y} &= x W^{[1]} W^{[2]} + b^{[1]} W^{[2]} + b^{[2]} \end{aligned}$$

$W' = W^{[1]} W^{[2]}$
 multiplier
 $(m, q) = (m, k) (k, q)$

$b' = b^{[1]} W^{[2]} + b^{[2]}$
 constant +
 $(1, q) = (1, k) (k, q) + (1, q)$

$$\hat{y} = x W' + b' \quad \text{shapes: } (1, q) = (1, m) (m, q) + (1, q)$$

e) Non linear activation functions are essential to give NN their representational power as without these activation functions, even if we have multiple NN layers like this example, they will be turned to one linear layer and will be equivalent to single linear transformation $\hat{y} = W'x + b$. By adding non-linear activation functions, NN model can learn complex boundaries.

(composition of linear maps is a linear map. A 3-layer network has same representation power of a single linear layer.)

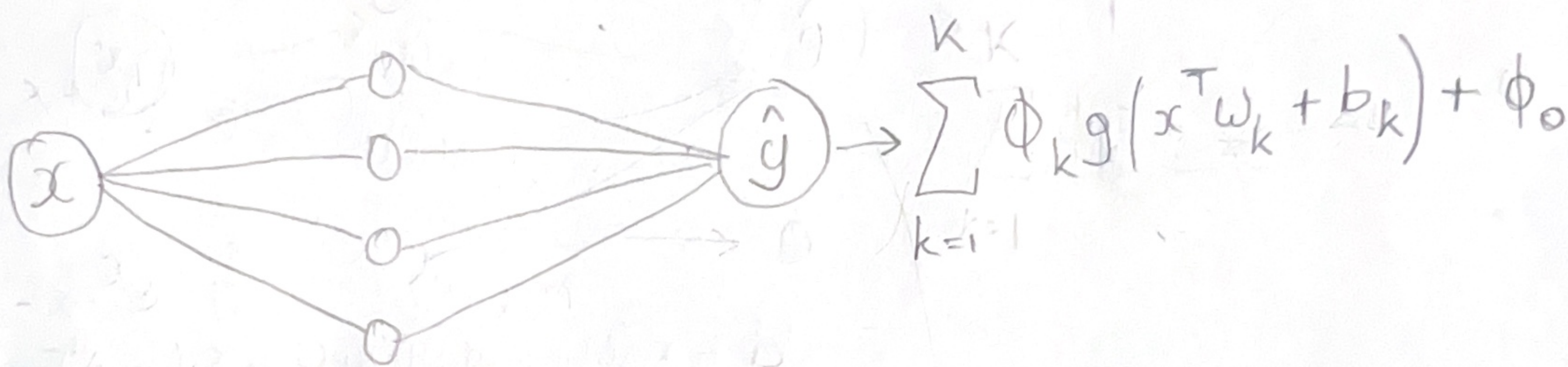
Problem 5 - Approximating a Function with Pulses

Pulse 1: $(s_1, s_2, h) = (0, 1, 3)$

Pulse 2: $(s_1, s_2, h) = (1, 2, 4)$

Pulse 3: $(s_1, s_2, h) = (2, 3, 4)$

Pulse 4: $(s_1, s_2, h) = (3, 4, 3)$



$\omega_1 = 0$	$b_1 = 1$	$\phi_1 = 3$	$\phi_0 = 0$
$\omega_2 = 0$	$b_2 = 1$	$\phi_2 = 1$	
$\omega_3 = 0$	$b_3 = 1$	$\phi_3 = 0$	
$\omega_4 = 0$	$b_4 = 1$	$\phi_4 = -1$	

Problem 6 - Stochastic Gradient Descent

i) $P(\text{not choosing a sample}) = \frac{n-1}{n} = 1 - \frac{1}{n}$

with replacement $\Rightarrow n$ independent events

$$\Rightarrow \left(1 - \frac{1}{n}\right) \times \dots \times \left(1 - \frac{1}{n}\right)$$

$$\Rightarrow \boxed{P_{\text{never}} = \left(1 - \frac{1}{n}\right)^n}$$

ii) $\lim_{x \rightarrow \infty} \left(1 + \frac{1}{x}\right)^x = e$; Show that $\lim_{n \rightarrow \infty} P_{\text{never}} \approx 0.3674$

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n$$

$$\text{let } y = \left(1 - \frac{1}{n}\right)^n$$

$$\ln(y) = n \ln\left(1 - \frac{1}{n}\right)$$

$$\lim_{n \rightarrow \infty} \ln(y) = \lim_{n \rightarrow \infty} \frac{\ln\left(1 - \frac{1}{n}\right)}{\frac{1}{n}} = \frac{\left(\frac{-1}{1 - \frac{1}{n}}\right)\left(\frac{1}{n^2}\right)}{-\frac{1}{n^2}} = -\frac{1}{1 - \frac{1}{n}} = -1$$

$$\Rightarrow \lim_{n \rightarrow \infty} \ln(y) = -1$$

$$\Rightarrow \boxed{\lim_{n \rightarrow \infty} y = e^{-1} = 1/e \approx 0.3679}$$

iii) One random observation $\text{Var}(\nabla f(x^{(i)}, y^{(i)})) = \sigma^2$

Sampling with replacement $\Rightarrow$ random variables are i.i.d

for k iid random variables with variance σ^2

$$\text{sample mean } \bar{X} = \frac{1}{k} \sum_{i=1}^k X_i$$

$$\boxed{\text{Var}(\bar{X}) = \frac{\sigma^2}{k}}$$

iv) The variance is reduced by a factor of k

v) Since the batch size is k , the variance of the gradient estimate is reduced by a factor of k . This means that the variance of the gradient estimate is smaller, which is good for convergence. However, the variance of the gradient estimate is still non-zero, and this may lead to the algorithm being noisy. One may want to reduce the variance by using a smaller batch size, but this may lead to the algorithm being more computationally expensive.

v) The claim in the paper is wrong because increasing the batch size and learning rate by a factor of k increases the variance by a factor of k

Let the variance of the gradient of one observation

$$\text{Var}(\nabla f(x^{(i)}, y^{(i)})) = \sigma^2$$

The variance of a mini batch of size k ,

$$\text{Var}(\nabla f(x, y)) = \frac{\sigma^2}{k} \text{ without changing the learning rate}$$

The variance of a mini batch of size k and increasing the learning rate by a factor of k

$$\text{Var}(k \nabla f(x, y)) = \frac{k^2 \sigma^2}{k} = k \sigma^2$$

To maintain a constant variance σ^2 , scale the batch size by k and scale the learning rate by $\sqrt{k}$.

AI Usage

universal approximation theorem

✓ Summarized the universal approximation theorem for neural networks



The **Universal Approximation Theorem (UAT)** states that a feedforward neural network with a **single hidden layer** and a finite number of neurons can approximate **any continuous function** on a compact subset of $\mathbb{R}^n$ to **arbitrary precision**, provided the activation function is non-polynomial (e.g., sigmoid, ReLU, tanh).

Key points:

- **First proved** by George Cybenko in 1989 (for sigmoid activations); extended by Hornik, Stinchcombe & White (1989) and Funahashi (1989). Leshno, Lin, Pinkus & Schocken (1993) generalized it to *any* non-polynomial activation.
- **Mechanism:** Universality is achieved by increasing the number of hidden neurons (network *width*). Separate results show that fixed-width but sufficiently *deep* networks are also universal.
- **Formal statement:** For any $f \in C([0, 1]^n)$ and any $\varepsilon > 0$, there exist weights w_j , biases b_j , and output coefficients a_j such that
$$\sup_{x \in [0, 1]^n} \left| f(x) - \sum_{j=1}^N a_j \sigma(b_j + w_j^\top x) \right| < \varepsilon.$$
- **Limitations:** The theorem is *existential* — it guarantees a network exists but gives no practical guidance on how to find the right architecture or train it. It also applies only to continuous functions and does not imply that a single network can approximate *all* functions simultaneously.